

# Knight

---

Imagine there exists an infinitely large chessboard.

We define coordinate  $(P, Q)$  as the square in row  $P$  and column  $Q$  in the chessboard.

Now, there's a [knight](#) in coordinate  $(X, Y)$  and he wishes to travel to coordinate  $(A, B)$ .

The knight hopes that he can reach the coordinate in **exactly**  $K$  steps. Can you determine if it is possible to do so?

If it is possible for the knight in  $(X, Y)$  to reach  $(A, B)$  in exactly  $K$  steps then output YES, otherwise output NO.

## Note

1. The chessboard is infinite in all directions, so the knight can also travel to negative coordinates.
2. When the knight is at coordinate  $(X, Y)$ , after one step it must move to one of the coordinates

```
(X + 2, Y + 1)
(X - 2, Y + 1)
(X + 2, Y - 1)
(X - 2, Y - 1)
(X + 1, Y + 2)
(X - 1, Y + 2)
(X + 1, Y - 2)
(X - 1, Y - 2)
```

(Just like the usual knight movement in chess) [Learn More](#)

## Input and Output Format

There are multiple test cases for each input this problem.

You are given  $T$  test cases in each input.

Each element in the list `cases` is an independent test case consisting of 5 integers

$K, X, Y, A, B$ .

Output  $T$  lines, where the  $i^{\text{th}}$  line is either YES or NO, the answer to the  $i^{\text{th}}$  test case.

I have written a Python code to convert the input to C++ friendly. You can check it out [here](#)

## Constraints

- $1 \leq T \leq 400$ ,
- $-1000 \leq X, Y, A, B \leq 1000$ ,
- $1 \leq K \leq 100000$ .

## Sample Input

```
T = 3
cases = [[2, 0, 0, 3, 3], [5, -2, -2, 100, 100], [2, 0, 0, 1, 2]]
```

## Sample Output

```
YES
NO
NO
```

## Explanation:

For the first test case,  $K = 2, X = 0, Y = 0, A = 3, B = 3$ .

A knight at  $(0, 0)$  can travel to  $(3, 3)$  in 2 steps by

$(0, 0) \rightarrow (1, 2) \rightarrow (3, 3)$

Therefore, the answer is YES.

For the second test case,  $K = 5, X = -2, Y = -2, A = 100, B = 100$ .

A knight at  $(-2, -2)$  cannot travel to  $(100, 100)$  in 5 steps.

Therefore, the answer is NO.

For the third test case,  $K = 2, X = 0, Y = 0, A = 1, B = 2$ .

A knight at  $(0, 0)$  cannot travel to  $(1, 2)$  in 2 steps.

Therefore, the answer is NO .

## Solution

---

Since the constraints are not big, we can picture it in a 4000x4000 grid. Denote  $steps(i, j)$  the minimum steps required to reach to  $(i, j)$  cell from  $(0, 0)$  in a two-dimension 4000 by 4000 vector/array, where  $0 \leq i, j \leq 4000$ . Denote the possible knight moves from  $(i, j)$  as the neighbours of  $(i, j)$ . 0 moves is required from  $(0, 0)$  to  $(0, 0)$ . From  $(0, 0)$ , the knight could only move to  $(1, 2)$  and  $(2, 1)$  in 1 step. From  $(1, 2)$ , the knight continues moves to  $(0, 4)$ ,  $(2, 4)$ ,  $(3, 3)$ ,  $(3, 1)$  in 2 steps (steps are counted from coordinate  $(0, 0)$ ). Notice that the knight could move back to  $(0, 0)$ , but we do not update  $steps(0, 0)$  since  $steps(0, 0)$  is 0, which is the minimum steps from  $(0, 0)$ . This pattern goes on until all  $(i, j)$  cells are filled up.

This solution is solved by well-known Breath-First Search algorithm which requires a queue for the coordinates. Queue has First-In-First-Out data structure. We put  $(0, 0)$  in our queue first. When  $(i, j)$  is the front of the queue, we update all the neighbours,  $steps(neighbour)$  by  $steps(i, j) + 1$ . If  $steps(neighbour)$  is less than or equal to  $steps(i, j) + 1$ , we do not update it. If the neighbour is updated, we place it at back of the queue. Then, we remove  $(i, j)$  from the queue after visiting all its neighbours.

After calculating all minimum steps required, we need to check is it possible to move from  $(X, Y)$  to  $(A, B)$  in  $K$  steps. Let us denote  $dx$  as the absolute value of the difference between the  $x$  coordinates,  $X$  and  $A$ ,  $dy$  as the absolute value of the difference between the  $y$  coordinates,  $Y$  and  $B$ . We could picture from  $(X, Y)$  to  $(A, B)$  as moving from  $(0, 0)$  to  $(dx, dy)$ . If  $K$  is less than  $steps(dx, dy)$ , it is impossible to reach to  $(dx, dy)$  in  $K$  steps. If  $K$  is equal to  $steps(dx, dy)$ , it is possible to reach to  $(dx, dy)$  in  $K$  steps. If  $K$  is more than  $steps(dx, dy)$ , we have  $K - steps(dx, dy)$  leftover steps after reaching  $(dx, dy)$ . We move back to  $(dx, dy)$  if  $K - steps(dx, dy)$  is even by moving to its neighbour and back to  $(dx, dy)$  (2 steps required each time). If  $K - steps(dx, dy)$  is odd, it is impossible to move back to  $(dx, dy)$ . This is because a knight will change the color of the square that it is located at when it moves a step. So it is impossible to get back the same color when you move in odd steps.

## Implementation

```

import queue

dx=[2,2,-2,-2,-1,1,-1,1]
dy=[1,-1,1,-1,-2,-2,2,2]

T = 3
cases = [[2, 0, 0, 3, 3], [5, -2, -2, 100, 100], [2, 0, 0, 1, 2]]
visited = [[-1 for i in range(2002)] for j in range(2002)]
q = queue.Queue()
q.put((0,0))
visited[0][0] = 0
while (not q.empty()):
    t = q.get()
    xt = t[0]
    yt = t[1]
    count = visited[xt][yt]
    for k in range(8):
        X = xt+dx[k]
        Y = yt+dy[k]
        #checking validity of neighbours
        if ((0 <= X and X < 2002) and (0 <= Y and Y < 2002)
            and (visited[X][Y] < 0)):
            visited[X][Y] = count + 1
            q.put((X,Y))

#(0,0) -> (-1,2) -> (1,1) required 2 steps
#previously doesn't consider negative coordinate
visited[1][1] = 2;
for z in range(T):
    k = cases[z][0]
    x = cases[z][1]
    y = cases[z][2]
    a = cases[z][3]
    b = cases[z][4]
    dx = abs(x - a)
    dy = abs(y - b)
    if (k - visited[dx][dy] >= 0 and (k - visited[dx][dy]) % 2 == 0):
        print("YES")
    else:
        print("NO")

```